Connections AI Solver

Studienarbeit

for the Informatik Department at the

DHBW Heidenheim

by

Ziyi Hong

TODO. TODO 2026

Processing period	January – March 2026, July – September 2026
Student ID	2694933
Course	Allgemeine Informatik
Evaluator	Tobias Heß

Declaration of Independence

I hereby declare that I have written my study on the topic "*Connections AI Solver*" independently and that I have not used any sources or aids other than the indicated ones. I also declare that the submitted electronic version matches the printed version.

MyPlace, July 11, 2026	
Place, Date	Signature

Abstract

Contents

List of Tables V

1 Introduction 1

1.1 Motivation . . . 1

1.2 Problem Statement . . . 2

1.3 Contribution . . . 3

1.4 Structure . . . 3

2 Foundations and Related Work 4

2.1 The NYT Connections Puzzle . . . 4

2.2 Relation Types and the Structure of Difficulty . . . 5

2.3 Challenges for Automated Solvers . . . 6

2.4 Small and Resource-Constrained Language Models as Solving Agents . . . 7

2.5 Text Embeddings and Clustering for Lexical Grouping . . . 8

2.6 Prompting Strategies for LLM Reasoning . . . 8

2.7 Related Work on NYT Connections Solvers . . . 9

3 Experimental Design and Implementation 12

4 Evaluation and Discussion 13

4.1 Experiments . . . 13

5 Conclusion and Future Work 14

List of Tables

2.1 Relation types in NYT Connections, with illustrative examples. 5

1 Introduction

Existing solvers for NYT Connections from fromer works rely heavily on large LLMs (see Section ??). They are limited by the cost or resources required to run large LLMs. And since this study is a student project with limited resources and budget, it focuses on small language models that can run locally. Small language models are more accessible and cost-effective, but they also struggle with reasoning tasks that require tracking global constraints across multiple words like solving Connections. This thesis investigates whether this reasoning gap can be narrowed not by scaling up the model, but by giving a small model additional, task-specific support.

1.1 Motivation

Solving Connections requires several distinct kinds of language understanding at once. Some groups are based on synonymy, while others may depend on wordplay, or knowledge of pop culture and named entities. The words given can be polysemy, which means words containing more than one meaning. Players need to use other words as the context to determine the meaning of the word. A single puzzle consists of several relation types and deliberately includes words that fit more than one plausible group, which is called "red herrings". Thus, it is not just about understanding the words themselves, but also about understanding how they relate to each other in a global context.

Large language models already achieve strong results on many natural language understanding tasks, but they are expensive to run and usually depends on cloud access. Small models are more accessible and cost-effective, but because of their limited capacity, they struggle with complicated reasoning tasks like solving Connections. Thus, using small models requires a good design of breaking down the big task into small tasks, and providing task specific support and prompting to the model, in oder to improve its performance. This study trys to analyze the possibility of solving Connections with a hybrid approach of llm and non-llm.

1.2 Problem Statement

Prior work on Connections solvers establishes two building blocks that this thesis builds on directly. Non-LLM baselines score candidate groups of four words using text embeddings and assemble a full solution through search or clustering. LLM-based approaches prompt a language model directly, sometimes guided by chain-of-thought or self-consistency prompting. Existing studies differ substantially in which embedding family they use and do not compare these choices under an otherwise fixed solver design.

They also treat the final assignment step with heuristics such as balanced clustering rather than an exact method. This step requires the sixteen words to split into exactly four groups of four. Finally, none of them evaluate solving accuracy separately for each difficulty tier. The puzzle's own difficulty labels suggest that categories differ systematically in the kind of reasoning they demand, a point discussed further in Chapter 2.

This thesis addresses this gap for a specific and practically relevant case. It centers on a small language model in the seven to eight billion parameter range, run locally rather than through a hosted API. It asks two research questions.

RQ1. To what extent does combining a small language model with embedding-based clustering and an exact constrained assignment step improve puzzle-solving accuracy compared to the same model used on its own?

RQ2. Does this improvement differ across difficulty tiers, in particular between the easier (yellow and green) and harder (blue and purple) categories?

It develops a hybrid solver that combines embedding-based clustering with LLM-based reasoning, relying only on small language models, for example Mistral 7b. The hybrid design combines the strengths of both methods to improve solving accuracy. The resulting solver aims to provide a lower-cost, more efficient alternative to large-model-dependent approaches.

The solver is evaluated on puzzles of varying difficulty levels, and the results are analyzed to assess the effectiveness of the hybrid approach. This thesis aims to show that even a puzzle as demanding as Connections can be solved with small language models when combined with embeddings, providing insight into the challenges that natural language processing and reasoning pose in the context of word puzzles.

1.3 Contribution

1.4 Structure

2 Foundations and Related Work

This chapter establishes the conceptual and technical foundation for this thesis. Section 2.1 introduces the NYT Connections puzzle and its rules. Section 2.2 develops a taxonomy of the semantic relations underlying puzzle categories and links it to the puzzle's difficulty structure. Section 2.3 discusses why the puzzle is difficult for automated solvers in general. Section 2.4 motivates the focus on small, locally deployable language models. Sections 2.5 and 2.6 cover the two technical building blocks of the proposed solver – text embeddings/clustering and LLM prompting strategies. Section 2.7 reviews prior solver designs for Connections and positions this thesis relative to them.

2.1 The NYT Connections Puzzle

NYT Connections is a daily word puzzle in which players must partition a set of sixteen words into four disjoint groups of four. Each correct group is assigned one of four color-coded difficulty labels – yellow (easiest), green, blue, and purple (hardest) – reflecting the puzzle's intended difficulty in recognizing the underlying grouping relation.

Two evaluation protocols are relevant for this thesis and are used interchangeably across the experiments described in **??**[1]:

- An **interactive setting** with a limited number of attempts (four in the original game) and "one-away" feedback: after each submitted group, the player learns whether the group was fully correct, wrong, or contained exactly three correct words ("one away").
- An **all-at-once setting** that requires predicting the complete 4×4 partition in a single submission, without intermediate feedback.

Solver communities have also adopted self-imposed variants to increase difficulty, such as attempting the purple category first ("purple-first"), which reverses the typical order of play by prioritizing the most abstract relation before the easier categories are identified. This practice illustrates that even human solvers treat the difficulty ordering yel-

[1] Adjust the chapter label `ch:methodology` once your methodology chapter exists.

low → green → blue → purple as a meaningful, exploitable structure rather than an arbitrary label – a point taken up again in Section 2.2.

2.2 Relation Types and the Structure of Difficulty

The categories in a Connections puzzle are not homogeneous in the kind of linguistic or world knowledge they require. Following the taxonomy summarized in Table 2.1, four broad relation types can be distinguished, with a fifth "mixed" category covering cases that combine more than one type.

Table 2.1: Relation types in NYT Connections, with illustrative examples.

Relation type	Example category and words
Semantic similarity	*FURTHEST POINT*: END, EXTREME, OPPOSITE, POLE [**nyt-puzzle-2026-01-05**]
Semantic relatedness	*THINGS YOU CAN DO ON SOCIAL MEDIA*: COMMENT, LIKE, LURK, POST [**nyt-puzzle-2026-01-05**]
Form- or rule-based relations	*ART MOVEMENTS, WITH "-ISM"*: BRUTAL, IMPRESSION, MANNER, REAL [**nyt-puzzle-2026-01-05**]
Referential relations	*RIHANNA #1 HITS*: DIAMONDS, SOS, UMBRELLA, WORK [**nyt-puzzle-2026-01-03**]
Mixed categories	*CAR BRAND HOMOPHONES*: INFINITY, MINNIE, OPAL, OUTIE [**nyt-puzzle-2025-12-29**]

Semantic similarity relations rest on near-synonymy or shared hypernymy (e.g., "types of flowers"); semantic relatedness relations hold between words that are not strictly similar but co-occur within a common theme or script; form- or rule-based relations are determined by surface regularities such as spelling, morphology, or conventional symbol meanings; referential relations require external world knowledge about named entities or factual lists. The hardest (purple) categories draw disproportionately on the less standard relation types – word and letter patterns such as rhymes, homophones, anagrams, or palindromes, unmarked fill-in-the-blank phrases, and pop-culture or proper-noun knowledge –

and frequently combine more than one relation type at once [**nyt-tips-tricks-2023**].

Working hypothesis adopted in this thesis. The color-coded difficulty ordering correlates, at least approximately, with this relation-type taxonomy: yellow and green categories tend toward semantic similarity and relatedness, which are directly addressable by distributional/embedding-based methods, whereas blue and purple categories skew toward form-based, referential, and mixed relations, which embeddings alone are ill-suited to capture and which instead require either symbolic pattern matching or LLM-internal world knowledge. This mapping is not drawn from a specific external source but is proposed here as an explanatory lens; it is revisited in **??**[2] as the basis for the per-tier error analysis.

2.3 Challenges for Automated Solvers

The relation-type diversity described above already indicates why Connections is difficult for automated approaches: a single puzzle may combine semantic similarity, semantic relatedness, form- or rule-based cues, and referential knowledge. Consequently, the task spans multiple NLP subproblems simultaneously – distributional semantics and sense disambiguation for similarity/relatedness categories, pattern matching for form-based categories, and retrieval of external, often culturally specific knowledge for referential categories – so that no single notion of similarity is sufficient across all groups.

The puzzle is also difficult by design. Categories frequently overlap in plausibility, creating red herrings: words that appear to fit one candidate grouping but ultimately belong elsewhere. Context clues are consequently critical for disambiguating individual words, and because the four groups must jointly partition all sixteen words, early (incorrect) commitments constrain what remains achievable for the rest of the puzzle [**nyt-tips-tricks-2023**].

Loredo Lopez, McDonald, and Emami [**loredolopez2025**] explicitly design their Connections benchmark to penalize fast, superficial-cue-driven ("System-1") guesses. This framing aligns with the shortcut-learning perspective: models can succeed by exploiting easy-to-use regularities rather than recovering the intended global grouping structure, a

[2] Adjust once your results chapter exists.

behavior expected whenever the data distribution offers shortcut opportunities and the evaluation objective rewards whatever features are most locally predictive [**geirhos2020**]. Connections' daily release cadence additionally supports robust, leakage-resistant model comparison, since each day's puzzle is unseen at evaluation time relative to a model's training cutoff [**loredolopez2025**].

2.4 Small and Resource-Constrained Language Models as Solving Agents

Most reported LLM results on Connections-style benchmarks use large, cloud-hosted models [**toddetal2024**, **loredolopez2025**, **zhangetal-connections**]. This thesis instead centers on small, locally executable models in the 7–8B parameter range (e.g., Mistral 7B, Llama 3.1 8B), run through a local inference server (Ollama) rather than a hosted API. Three considerations motivate this focus:

1. **Deployment constraints.** Local execution removes dependence on external API availability, per-token cost, and data egress, which is relevant whenever a solver is meant to run on constrained or offline infrastructure.

2. **The capability gap as the actual object of study.** Because small models trail larger ones on complex reasoning tasks, they expose failure modes – shortcut reliance, weak constraint tracking across a 16-word global partition, limited world knowledge for referential categories – more readily than large models do. This makes small models a more informative testbed for evaluating whether *external scaffolding* (embeddings, clustering, structured verification) can substitute for missing model-internal capability, which is the central question this thesis investigates.

3. **Comparability to prior work.** Loredo Lopez, McDonald, and Emami [**loredolopez2025**] already evaluate multiple LLMs of varying size under different prompting schemes on Connections, which provides a point of comparison for the relative performance of the small models used here (Section 2.7).

2.5 Text Embeddings and Clustering for Lexical Grouping

At the representation level, Connections is closely tied to text embeddings, semantic similarity, and clustering. Early work on distributed word representations introduced dense word vectors capturing lexical regularities through co-occurrence statistics [**mikolov2013**]. Subsequent surveys trace the shift from such static word vectors to contextualized representations, including sentence-level embeddings optimized for semantic similarity [**chandrasekaran2021**
transformer-based embedding models are now the standard approach for encoding context-dependent lexical and semantic relations [**incitti2023**], and a broader survey perspective summarizes representation choices and their implications for similarity modeling across NLP tasks more generally [**patil2023**].

Classical clustering algorithms – k-means, k-means++, and DBSCAN, together with keyword-clustering techniques – group items based on distances within such embedding spaces [**ester1996dbscan**, **arthur2007kmeanspp**]. More recent work shows that LLM-derived embeddings can improve resulting cluster structure, and that clustering itself can be reframed as an in-context classification problem solved directly by an LLM rather than by a separate distance-based algorithm [**huang2025textclustering**, **petukhova2025**].

This thesis uses SentenceTransformer-style encoders (specifically `BAAI/bge-large-en-v1.5`, alongside comparison runs with other encoder families such as the E5 series and MiniLM) as the embedding backbone for the non-LLM baseline and for the clustering pre-processing stage of the hybrid pipeline (**??**). The choice of embedding family is treated as an experimental variable rather than fixed a priori, following the observation in Section 2.7 that prior work on Connections instantiates broadly comparable solver structures with substantially different embedding families, without directly comparing them under a controlled setup.

2.6 Prompting Strategies for LLM Reasoning

Chain-of-thought (CoT) prompting elicits intermediate reasoning steps from an LLM before it commits to a final answer and has been shown to improve performance on complex, multi-step tasks [**wei2022cot**]. Prompting-technique reference guides collect CoT and related variants in a practitioner-oriented form [**promptguide-cot**].

The solver pipelines evaluated in this thesis instantiate several points along this spectrum: a zero-shot condition, a few-shot condition using a single worked example, and a CoT condition that decomposes reasoning into an explicit analyze–group–verify sequence (see `system_prompt()` in the implementation). A self-consistency variant, which samples multiple reasoning traces and aggregates over them rather than committing to a single greedy completion, is evaluated as an additional condition. Loredo Lopez, McDonald, and Emami [**loredolopez2025**] similarly contrast direct input–output prompting against CoT and self-consistency variants on Connections specifically, which this thesis uses as a point of reference in Section 2.7 and **??**.

2.7 Related Work on NYT Connections Solvers

Three prior studies directly inform the solver design used in this thesis.

Todd et al. [toddetal2024] study Connections using both embedding-based heuristics and prompted LLMs. Their non-LLM baseline encodes words with SentenceTransformer-style models, exhaustively scores all $\binom{16}{4}$ candidate four-word groups by within-group cosine-similarity cohesion, and constructs a solution iteratively; they also consider an all-at-once variant that ranks complete 4×4 partitions by summed cohesion.

Loredo Lopez, McDonald, and Emami [loredolopez2025] propose a "System-1-like" heuristic baseline using Multilingual-E5-Large-Instruct embeddings, scoring candidate groups with a discriminative function

$$S = G - P, \tag{2.1}$$

where G aggregates multiple within-group cohesion signals and P penalizes groups that are insufficiently distinctive from the remaining words. Full solutions are assembled via beam search. They evaluate multiple LLMs under direct, chain-of-thought, and self-consistency prompting schemes.

Correspondence to this project's implementation. The `score_group()` function in `herustic_solver.py` computes

$$G = 0.4 \cdot I + 0.3 \cdot s + 0.3 \cdot V$$

(a weighted combination of negative inertia, minimum pairwise cosine similarity, and a mean/variance cohesion term) and a final score $S = G - P$, assembled via a beam-search procedure over group choices – structurally identical to the scoring function in Equation 2.1 as described by Loredo Lopez et al. [**loredolopez2025**]. **This correspondence must be stated explicitly in the methodology chapter** as the basis for the reproduced baseline, both to satisfy the "solides Fundament" principle of the DHBW guideline and to avoid any appearance of an unattributed structural borrowing.

Zhang, Jiang, and Ye [zhangetal-connections] frame the task as equal-size constrained clustering and apply Balanced k-means on static word embeddings (GloVe/-word2vec), with an alternative KMeans-plus-rebalancing baseline to enforce the four-groups-of-four partition. They additionally explore LLM-based pipelines including in-context learning, parameter-efficient fine-tuning (QLoRA), rationale distillation from a larger teacher model, and feedback-driven refinement.

Synthesis and positioning of this thesis. These three studies converge on a common structural insight – that a Connections solver decomposes into (a) a representation/scoring stage over candidate groups and (b) an assignment stage that resolves the equal-size global partition – while differing substantially in how each stage is instantiated: embedding family (E5 [**loredolopez2025**] vs. SentenceTransformer/MPNet [**toddetal2024**] vs. static word2vec/GloVe [**zhangetal-connections**]), scoring function, and assignment method (iterative greedy construction [**toddetal2024**], beam search [**loredolopez2025**], balanced/rebalanced k-means [**zhangetal-connections**]). None of the three directly compares embedding families under an otherwise fixed solver structure, and only Zhang, Jiang, and Ye [**zhangetal-connections**] treat the assignment stage as an explicitly constrained combinatorial problem – but resolve it with a clustering heuristic rather than an exact method.

This thesis addresses both gaps directly. First, it reproduces the [**loredolopez2025**] scoring function as a fixed non-LLM baseline and instantiates it with multiple embedding families under an otherwise controlled setup (**??**), extending the embedding-family comparison that [**toddetal2024**, **loredolopez2025**, **zhangetal-connections**] motivate but do not themselves carry out. Second, building on the constrained-clustering framing of [**zhangetal-connections**], it replaces the balanced/rebalanced clustering heuristic with exact and near-exact constrained-assignment methods (dynamic programming and integer linear programming via OR-Tools) applied to LLM- and embedding-scored candidate groups, combined with an explicit LLM-based trap/red-herring verification stage – directly addressing the red-herring challenge identified in Section 2.3 [**nyt-tips-tricks-2023**]. Third, unlike [**toddetal2024**, **loredolopez2025**, **zhangetal-connections**], which evaluate primarily at the level of overall puzzle success, this thesis evaluates performance separately per difficulty tier (Section 2.2), which is necessary to assess the specific research question of whether the proposed hybrid pipeline improves solving accuracy on the easier (yellow/green) categories relative to the small-model baselines established in Section 2.4.

3 Experimental Design and Implementation

4 Evaluation and Discussion

4.1 Experiments

How do we build it: data, pipeline, modules.

- Pure embedding + clustering solver
- Pure LLM solver
- Embedding + LLM solver
- Improvement

5 Conclusion and Future Work